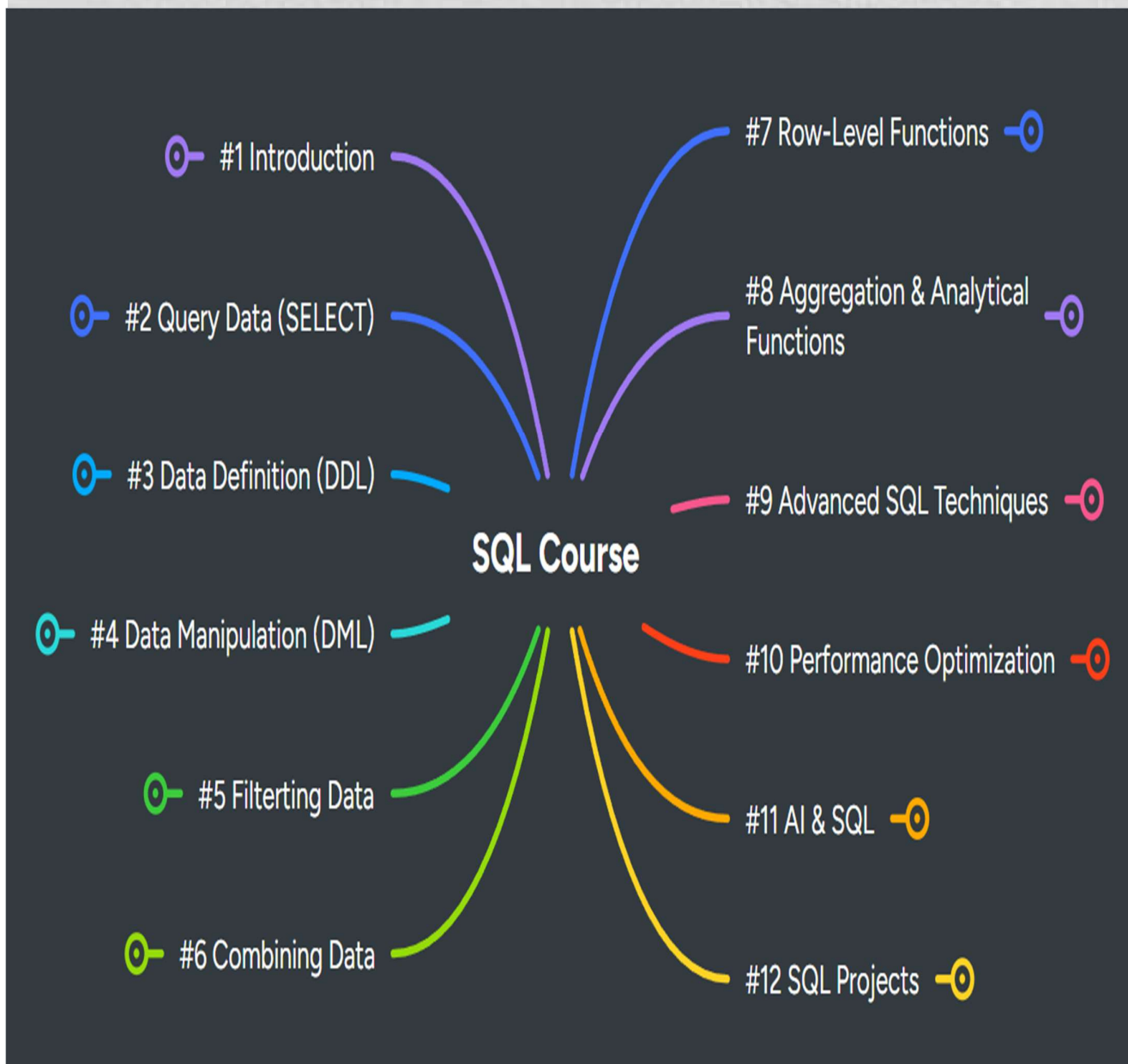


SQL ROADMAP



DataBase

What is Data?

- ✓ Data is a row fact which is used to describe the attribute of an entity.

What is DataBase?

- ✓ It is Organized collection of data so that it can be easily accessed.

The basic operation we can program our database are CURD operations.

- ✓ C :- Create
- ✓ U :- Read
- ✓ R :- Update
- ✓ D :- delete

Function Of DataBase

1. **Documents:-** Store data as document.
Ex:- Mongodb, CouchDB
2. **Graph:-** Store data as nodes and edges
Ex:- Neo4j
3. **Column-Based:-** Store data in columns instead of row.
Ex:- Apache cassandra, Amazon Redship
4. **Key-value:-** data is stored as key-Value Pairs
Ex:- Reais, Amazon DynamoDB

2. DBMS(Database Management System)

- ✓ DBMS is software used to create, store, manage, and control databases.

Functions of DBMS

1. **Data Storage**
Stores large amounts of data in databases.
2. **Data Retrieval**
Allows users to access data easily.
3. **Data Security**
Protects data from unauthorized access.
4. **Data Consistency**
Ensures accurate and reliable data.
5. **Data Backup and Recovery**
Helps recover data if it is lost.

3. RDBMS (Relational Database Management System)

- ✓ RDBMS is software used to create, manage, and maintain relational databases.

4. Type of DataBase

- ✓ There are two type of DataBase

1. Non-Relational DataBase

- ✓ A Non-Relational Database is a type of database that does not store data in tables with rows and columns like relational databases.

Ex:- MongoDB

Example (Document Format)

```
</> JSON  
  
{  
  "id": 1,  
  "name": "Amit",  
  "age": 22,  
  "course": "BCA"  
}
```

2. Relational DataBase

- ✓ A Relational Database is a type of database that stores data in tables made of rows and columns.

Ex:- MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, SQLite

Create Database

SQL command used to create a database

Syntax

```
CREATE DATABASE database_name
```

Show Databases

Used to display all databases.

Syntax

```
SHOW DATABASES
```

Use Database

Used to select a database for working.

Syntax

```
USE database_names;
```

Delete Database

Used to remove a database.

Syntax

```
DROP DATABASE database_name;
```

SQL(Structured Query Language)

What is Sql.

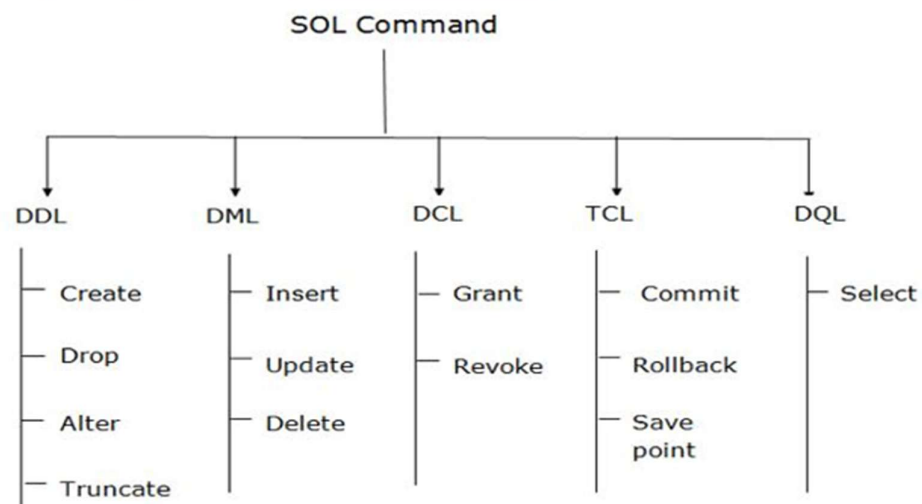
- ✓ **SQL (Structured Query Language)** is a programming language used to **communicate with databases**.

Why Learn SQL.

- ✓ Ubiquitous in Companies
- ✓ Essential for Data Analysis
- ✓ Handles Large Data
- ✓ Business Analyst

1. Types of SQL Commands

There are five types of SQL commands:



i. DDL(Data Definition language)

- ✓ Used to define and manage database structures like databases and tables.

Common DDL Commands:

1) **CREATE** – create new objects in a database

Example

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Grade VARCHAR(10)  
);
```

2) **ALTER** – Modify existing database objects

Example

a) Add a Column

```
ALTER TABLE Students  
ADD Email VARCHAR(100);
```

b) Modify a Column

```
ALTER TABLE std  
ALTER COLUMN name VARCHAR(100);
```

c) EXEC a Table

```
EXEC sp_rename 'std', 'students';
```

d) Rename column

```
EXEC sp_rename 'std.grade', 'student_grade', 'COLUMN';
```

3) **DROP** – Delete database objects

a) Drop a Table

```
DROP TABLE SchoolStudents;
```

b) Drop a Database

```
DROP DATABASE SchoolDB;
```

- ✓ Deletes the database and all its tables permanently.

c) Drop a column

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

4) **TRUNCATE** – Remove all records from a table

```
TRUNCATE TABLE Students;
```

✓ Removes all rows from the table, but the table structure remains intact.

ii. **DML(Data Manipulation Language)**

✓ DML is used to manipulate the data inside database tables, not the structure

1) **INSERT** – Add new data into a table

Syntax:

```
INSERT INTO table_name (column1, column2, ...)  
VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO Students (ID, Name, Age, Grade)  
VALUES (1, 'Suraj Kumar', 18, '12th');
```

2) **UPDATE** – Modify existing data

✓ Change data in one or more rows of a table.

Syntax:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Example

```
UPDATE Students  
SET Age = 19  
WHERE ID = 1;
```

3) **DELETE** – Remove rows from a table

✓ Delete one or more rows from a table.

Syntax

```
DELETE FROM table_name  
WHERE condition;
```

Example

```
DELETE FROM Students  
WHERE ID = 1;
```

Ya

```
DELETE FROM Students; -- removes all rows, table stays
```

iii. DCL (Data Control Language)

- ✓ DCL is a type of SQL command used to control access and permissions for users in a database.

1). GRANT – Give Permissions

Syntax

```
GRANT privilege1, privilege2  
ON table_name  
TO user_name;
```

Example

```
GRANT SELECT, INSERT, UPDATE  
ON Students  
TO 'suraj_user';
```

2). REVOKE – Remove Permissions

Syntax

```
REVOKE privilege1, privilege2  
ON table_name  
FROM user_name;
```

Example

```
REVOKE INSERT, UPDATE  
ON Students  
FROM 'suraj_user';
```

iv. TCL (Transaction Control Language)

- ✓ TCL is a type of SQL command used to manage transactions in a database.

1) COMMIT – Save Changes

- ✓ Save all changes made in the current transaction permanently

Syntax

```
COMMIT;
```

Example

```
INSERT INTO Students (ID, Name, Age) VALUES (2, 'Amit', 17, '11th');  
COMMIT;
```

2) ROLLBACK – Undo Changes

- ✓ Undo changes made in the current transaction

Syntax

```
ROLLBACK;
```

Example

```
INSERT INTO Students (ID, Name, Age) VALUES (3, 'Riya', 16, '10th');  
ROLLBACK;
```

3) SAVEPOINT – Partial Rollback

- ✓ Set a point within a transaction to which you can later roll back

Syntax

```
SAVEPOINT savepoint_name;
```

Example

```
INSERT INTO Students (ID, Name, Age) VALUES (4, 'Rahul', 18, '12th');  
SAVEPOINT sp1;  
  
INSERT INTO Students (ID, Name, Age) VALUES (5, 'Neha', 17, '11th');  
  
ROLLBACK TO sp1;
```

v. DQL (Data Query Language)

- ✓ DQL is a type of SQL command used to retrieve or query data from a database.

1) SELECT – Retrieve Data

- ✓ SELECT is used to fetch data from one or more tables. You can choose specific columns, all columns, filter data, sort it, or even do calculations

Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
ORDER BY column ASC|DESC;
```


Example

a) Select all column

- ✓ Selects all columns from a table.

```
SELECT * FROM Students;
```

b) Select specific columns

- ✓ Selects only the columns you want.

```
SELECT column1, column2 FROM Students;
```

1. FROM

- ✓ Specifies the table from which data is selected.

Syntax

```
SELECT column_name  
FROM table_name;
```

2. WHERE

- ✓ filters records based on a **specific condition**.
- ✓ Apply conditions like =, >, <, LIKE, IN

Syntax

```
SELECT column_name  
FROM table_name  
WHERE condition;
```

3. ORDER BY

- ✓ sorts the result set in **ascending or descending order**.

Syntax

```
SELECT column_name  
FROM table_name  
ORDER BY column_name ASC | DESC;
```

4. HAVING

- ✓ HAVING is used to **filter grouped data** after GROUP BY.
- ✓ Apply conditions on aggregated results

Syntax

```
SELECT column_name, aggregate_function(column)
FROM table_name
GROUP BY column_name
HAVING condition;
```

5. GROUP BY

- ✓ GROUP BY groups rows that have the **same values in specified columns**.
- ✓ Used with aggregate functions

Syntax

```
SELECT column_name, aggregate_function(column)
FROM table_name
GROUP BY column_name;
```

6. DISTINCT

- ✓ returns **unique values only** from a column.
- ✓ Remove duplicate records

Syntax

```
SELECT DISTINCT column_name
FROM table_name;
```

7. TOP

- ✓ limits the number of rows returned in the result.
- ✓ Display only a specific number of records
- ✓ Improve performance for large datasets

Syntax

```
SELECT TOP number column_name
FROM table_name;
```


Constraints

1. Primary Key

A Primary Key is a column (or set of columns) that uniquely identifies every record in a table.

Syntax

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

2. Foreign Key

A Foreign Key is a column in one table that references the Primary Key of another table.

Syntax

```
CREATE TABLE orders (  
    order_id INT,  
    customer_id INT,  
    FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

3. NOT NULL

NOT NULL ensures that a column cannot store NULL (empty) values.

Syntax

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(50) NOT NULL  
);
```

4. UNIQUE

UNIQUE ensures that all values in a column are different.
No duplicate values are allowed.

Syntax

```
CREATE TABLE users (  
    id INT,  
    email VARCHAR(100) UNIQUE  
);
```

5. CHECK

The CHECK constraint ensures that values in a column satisfy a specific condition.

Syntax

```
CREATE TABLE employee (  
    id INT,  
    salary INT CHECK (salary > 0)  
);
```

6. DEFAULT

The DEFAULT constraint sets a default value for a column if no value is provided.

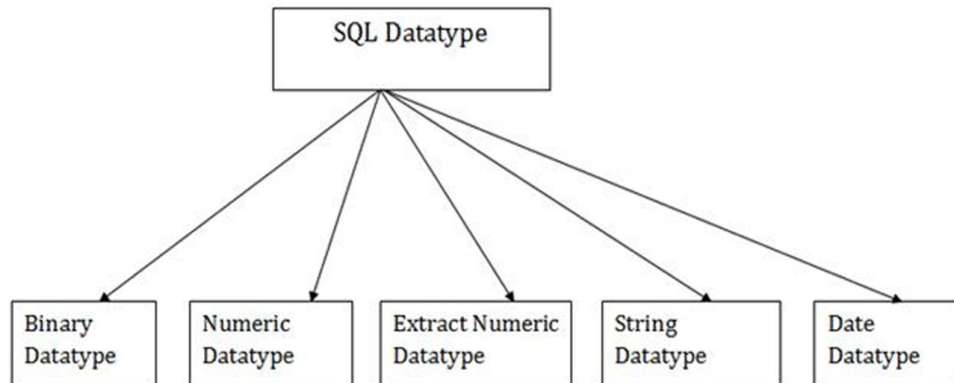
Syntax

```
CREATE TABLE students (  
  id INT,  
  city VARCHAR(50) DEFAULT 'Delhi'  
);
```

Datatype

What is SQL Datatype?

- ✓ SQL Data Type is used to define the type of data that a column can store in a table.



1. Binary Datatype

- ✓ Binary data types are used to store binary data (0 and 1) such as images, files, audio, or encrypted data.

BINARY

Stores **fixed-length** binary data.

Syntax

```
column_name BINARY(n)
```

VARBINARY

Stores variable-length binary data.

Syntax

```
column_name VARBINARY(n)
```

BLOB (Binary Large Object)

Used to store large binary files like images, videos, and audio.

Syntax

```
column_name BLOB
```

2. Numeric Datatype

- ✓ Numeric data types are used to store numeric values (numbers) in a database.
- ✓ These can be **whole numbers (integers)** or **decimal numbers**.

INT (Integer)

INT is used to store **whole numbers** (without decimal points).

Range:

-2,147,483,648 to 2,147,483,647

Syntax

```
INT
```

SMALLINT

SMALLINT stores smaller whole numbers than INT.

Range

-32,768 to 32,767

Syntax

```
SMALLINT
```

3. Exact Numeric Datatype

Exact Numeric Data Types are used to store numbers with exact precision.

DECIMAL

DECIMAL stores **exact decimal numbers** with fixed precision and scale

Syntax

```
DECIMAL(precision, scale)
```

- **Precision** → Total number of digits
- **Scale** → Number of digits after the decimal point

NUMERIC

NUMERIC is similar to DECIMAL. It stores exact numeric values with precision and scale.

Syntax

```
NUMERIC(precision, scale)
```

4. String Datatype

- ✓ String datatypes store text or character data.

CHAR

CHAR is used to store **fixed-length characters**.

Range

0 to 255 characters

Syntax

```
CHAR(size)
```

VARCHAR

VARCHAR is used to store **variable-length characters**.
It only stores the **actual characters**, not extra spaces.

Range

0 to 65,535 characters

Syntax

```
VARCHAR(size)
```

TEXT

TEXT is used to store **large amounts of text data**.

Range

Up to 65,535 characters

Syntax

```
TEXT
```

5. Date Datatype

- ✓ Date datatypes store **date and time values**.

DATE

TIME is used to store only the time (hours, minutes, seconds).

Format:

YYYY-MM-DD

Example:

```
CREATE TABLE students (  
    id INT,  
    dob DATE  
);
```

TIME

TIME is used to store **only the time** (hours, minutes, seconds).

Format:

HH:MM:SS

Example

```
CREATE TABLE meeting (  
    meeting_time TIME  
);
```

DATETIME

DATETIME stores **both date and time**.

Format

YYYY-MM-DD HH:MM:SS

Example

```
CREATE TABLE orders (  
    order_date DATETIME  
);
```

TIMESTAMP

TIMESTAMP also stores date and time, but it is mainly used to record the exact time when a row is created or updated automatically.

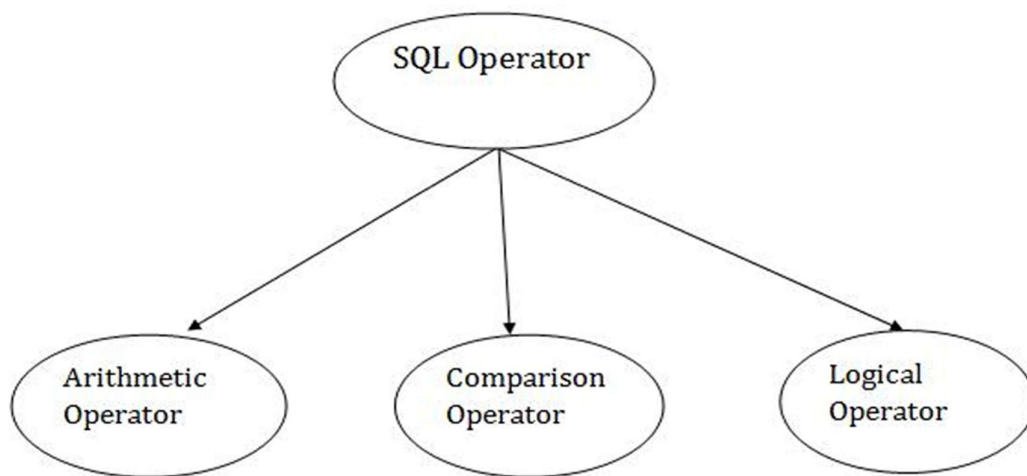
Format

YYYY-MM-DD HH:MM:SS

Example

```
CREATE TABLE logs (  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

OPERATORS



1. Arithmetic Operator

Operator	Description
+	It adds the value of both operands.
-	It is used to subtract the right-hand operand from the left-hand operand.
*	It is used to multiply the value of both operands.
/	It is used to divide the left-hand operand by the right-hand operand.
%	It is used to divide the left-hand operand by the right-hand operand and returns reminder.

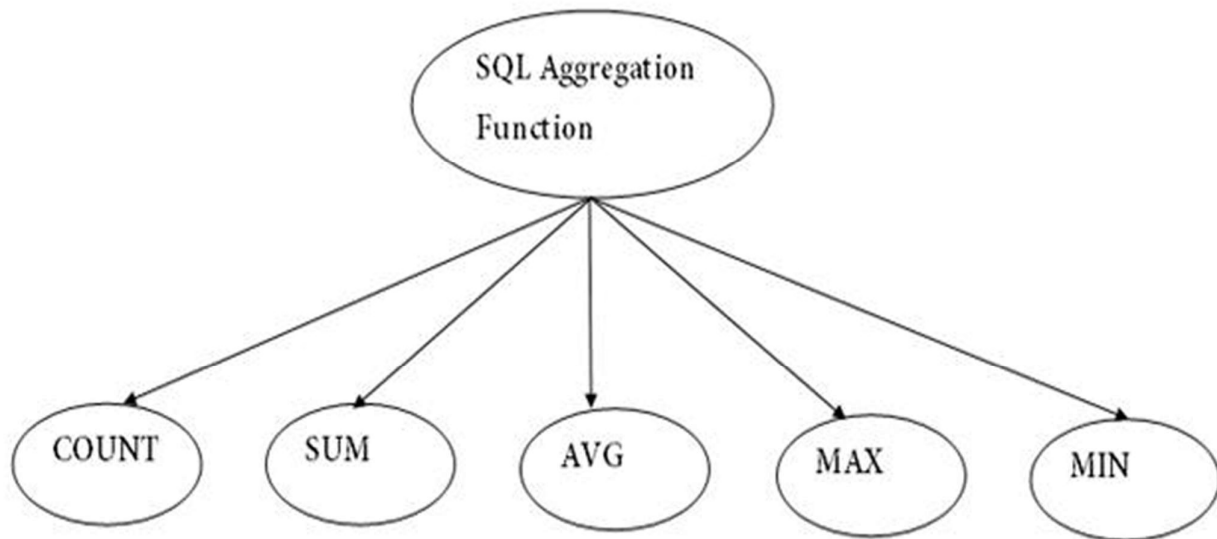
2. Comparison Operators

Operator	Description
=	It checks if two operands values are equal or not, if the values are equal then condition becomes true.
!=	It checks if two operands values are equal or not, if values are not equal, then condition becomes true.
<>	It checks if two operands values are equal or not, if values are not equal then condition becomes true.
>	It checks if the left operand value is greater than right operand value, if yes then condition becomes true.
<	It checks if the left operand value is less than right operand value, if yes then condition becomes true.
>=	It checks if the left operand value is greater than or equal to the right operand value, if yes then condition becomes true.
<=	It checks if the left operand value is less than or equal to the right operand value, if yes then condition becomes true.
!<	It checks if the left operand value is not less than the right operand value, if yes then condition becomes true.
!>	It checks if the left operand value is not greater than the right operand value, if yes then condition becomes true.

3. Logical Operators

Operator	Description
All	It compares a value to all values in another value set.
AND	It allows the existence of multiple conditions in an SQL statement.
ANY	It compares the values in the list according to the condition.
Between	It is used to search for values that are within a set of values.
IN	It compares a value to that specified list value.
NOT	It reverses the meaning of any logical operator.
OR	It combines multiple conditions in SQL statements.
EXIST	It is used to search for the presence of a row in a specified table.
LIKE	It compares a value to similar values using wildcard operator.

Aggregate Functions



What are SQL Aggregation Functions

- ✓ Aggregation functions in SQL are used to perform calculations on a set of values and return one result.
- ✓ They are commonly used with GROUP BY to summarize data.

Types of SQL Aggregation Functions

1. Count

COUNT() is used to count the number of rows in a table or column.

Syntax

```
SELECT COUNT(column_name)
FROM table_name;
```

2. SUM

SUM() is used to add all values in a numeric column.

Syntax

```
SELECT SUM(column_name)
FROM table_name;
```

3. AVG

AVG() calculates the average value of a numeric column.

Syntax

```
SELECT AVG(column_name)
FROM table_name;
```

4. MAX

MAX() returns the largest value in a column.

Syntax

```
SELECT MAX(column_name)
FROM table_name;
```

5. MIN

MIN() returns the smallest value in a column.

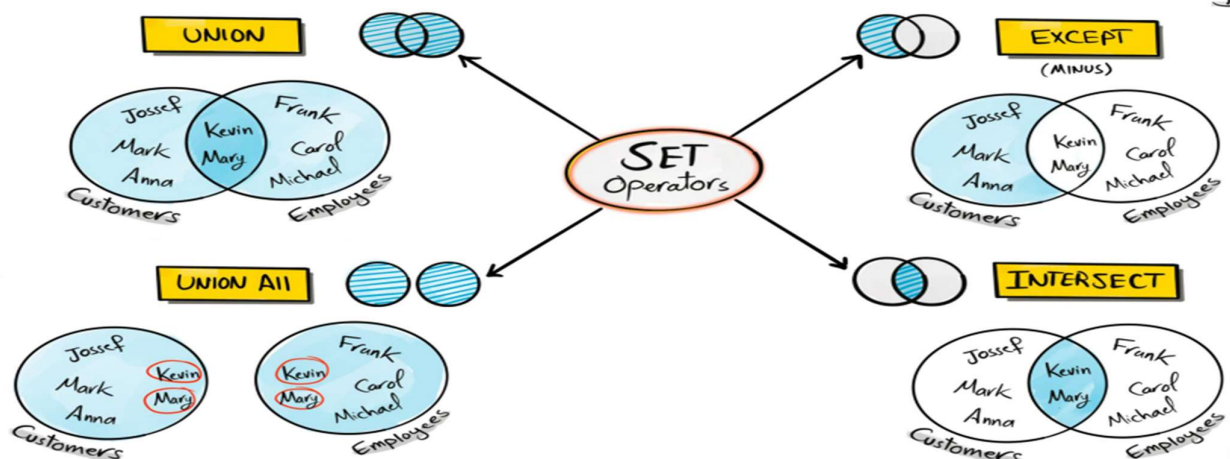
Syntax

```
SELECT MIN(column_name)
FROM table_name;
```

Combine Table

Set Operators in SQL

Set Operators are used to combine the result of two or more SELECT queries into a single result.



Types of Set Operators

1. Union

UNION combines the results of two queries and **removes duplicate rows**.

Syntax

```
SELECT column1, column2
FROM table1
UNION
SELECT column1, column2
FROM table2;
```

2. UNION ALL

UNION ALL combines the results of two queries but keeps duplicate rows.

Syntax

```
SELECT column1
FROM table1
UNION ALL
SELECT column1
FROM table2;
```

3. INTERSECT

INTERSECT returns only the common rows from both queries.

Syntax

```
SELECT column1
FROM table1
INTERSECT
SELECT column1
FROM table2;
```

4. EXCEPT (or MINUS in Oracle)

EXCEPT returns rows from the first query that are not in the second query.

Syntax

```
SELECT column1
FROM table1
EXCEPT
SELECT column1
FROM table2;
```


JOIN IN SQL

What is Join?

Join is used to combine rows from two or more table based on a related column
Usually the relationship is created using **Primary Key and Foreign Key**.

Syntax

```
SELECT column1, column2
FROM table1
JOIN table2
ON table1.common_column = table2.common_column;
```

Explanation

SELECT → columns select karne ke liye

FROM → first table

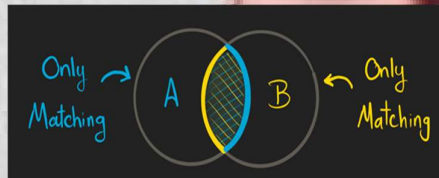
JOIN → second table

ON → condition jisse tables connect hoti hain

Types of JOIN in SQL

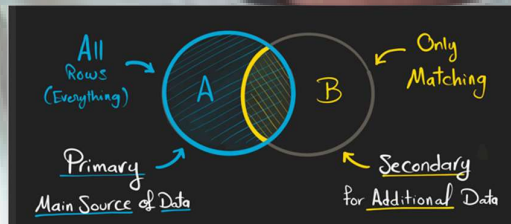
1. INNER JOIN

Returns only the rows that have matching values in both tables.



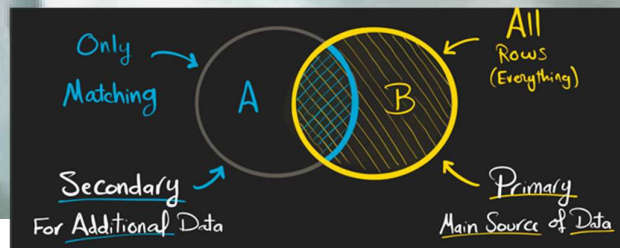
2. LEFT JOIN (LEFT OUTER JOIN)

Returns **all rows from the left table** and matching rows from the right table.
If there is no match, **NULL** values are returned.



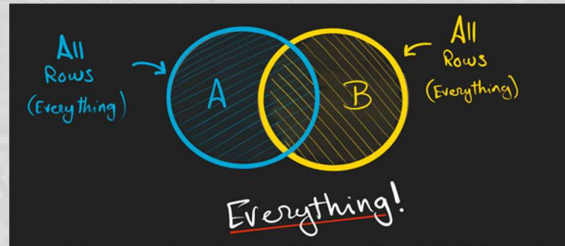
3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table.



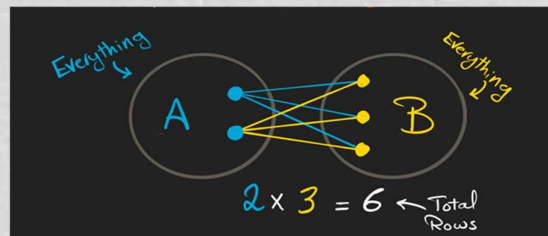
4. FULL JOIN (FULL OUTER JOIN)

Returns **all rows from both tables**, whether there is a match or not.



5. CROSS JOIN

A SELF JOIN joins a table to itself using a related column.



6. SELF JOIN

A CROSS JOIN returns the Cartesian product of two tables.

Subquery

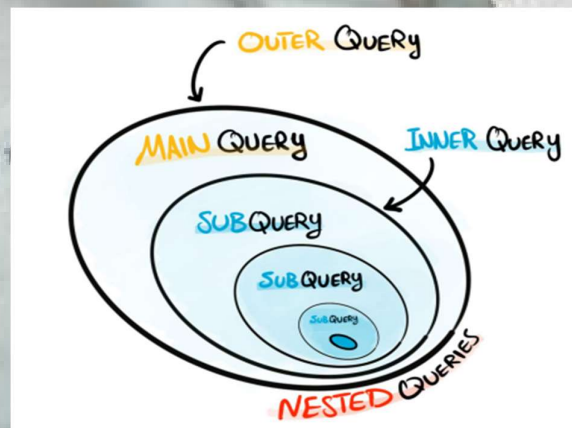
What is Subquery?

A subquery is a query inside another query.

A subquery is a nested query written inside SELECT, INSERT, UPDATE, or DELETE statements.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name = (
  SELECT column_name
  FROM table_name
  WHERE condition
);
```



Types of Subqueries

1. Single Row Subquery

A **Single Row Subquery** is a subquery that **returns only one row (one value)**.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator (
    SELECT column_name
    FROM table_name
    WHERE condition
);
```

Operator

- =
- >
- <
- >=
- <=
- <>

Example

```
SELECT *
FROM students
WHERE marks = (
    SELECT MAX(marks)
    FROM students
);
```

2. Multiple Row Subquery

A **Multiple Row Subquery** is a subquery that **returns more than one row (multiple values)**.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN (
    SELECT column_name
    FROM table_name
    WHERE condition
);
```

Operator

IN	Match any value in list
ANY	Greater/less than at least one value
ALL	Greater/less than all values
EXISTS	Checks if rows exist

Example

```
SELECT name, marks
FROM students
WHERE marks > ALL (
    SELECT marks
    FROM students
    WHERE marks > 80
);
```

3. Correlated Subquery

A **Correlated Subquery** is a subquery that **depends on the outer query**.

Syntax

```
SELECT column_name
FROM table1 t1
WHERE column_name operator (
    SELECT column_name
    FROM table2 t2
    WHERE t1.column = t2.column
);
```

Example

```
SELECT name, salary, department
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e1.department = e2.department
);
```

4. Subquery in SELECT

The subquery is inside the SELECT clause and returns a single value for each row.

Syntax

```
SELECT column,  
      (SELECT column FROM table WHERE condition) AS alias  
FROM table;
```

Example

```
SELECT name,  
      (SELECT AVG(marks) FROM Marks) AS avg_marks  
FROM Students;
```

5. Subquery in FROM

The subquery is used as a filter condition.

Syntax

```
SELECT column  
FROM (SELECT column FROM table WHERE condition) AS temp_table;
```

Example

```
SELECT subject, avg_marks  
FROM (  
    SELECT subject, AVG(marks) AS avg_marks  
    FROM Marks  
    GROUP BY subject  
    ) AS subject_avg;
```

6. Subquery in WHERE

The subquery acts as a temporary table that the main query can use.

Syntax

```
SELECT column  
FROM table  
WHERE column operator (SELECT column FROM table WHERE condition);
```


Example

```
SELECT name
FROM Students
WHERE marks > (
    SELECT AVG(marks)
    FROM Students
);
```

CASE STATEMENT

The CASE statement in SQL is a conditional expression used to perform IF-ELSE like logic inside a query.

Why WE USED CASE STATEMENT?

- To apply conditions in SQL queries
- To categorize data (like grades, salary levels, status, etc.)
- To make queries more dynamic and flexible

Syntax

1. SIMPLE CASE

```
CASE expression
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ELSE default_result
END
```

2. Searched CASE

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

3. Example

```
SELECT name, salary,
CASE
  WHEN salary >= 5000 THEN 'High Salary'
  WHEN salary >= 3000 THEN 'Medium Salary'
  ELSE 'Low Salary'
END AS salary_category
FROM Employee;
```

SQL Functions

What are SQL Functions

SQL Functions are used to perform calculations or operations on data and return a result.

- Aggregate data
- Transform data
- Calculate values

Types of SQL Functions

1. Aggregate Functions

Aggregate functions **perform calculations on a group of rows.**

Function	Meaning
COUNT()	Count rows
SUM()	Total value
AVG()	Average value
MAX()	Highest value
MIN()	Lowest value

Syntax

```
SELECT FUNCTION_NAME(column_name)
FROM table_name;
```

2. Single Row Functions

Single row functions **operate on one row and return one result per row.**

Types include:

Type	Example
String Functions	UPPER(), LOWER(), LENGTH()
Numeric Functions	ROUND(), ABS()
Date Functions	NOW(), CURDATE()

I. String Function Examples

String Functions are used to manipulate text or character data.

1. UPPER()

Convert names to uppercase.

2. LOWER()

Convert names to lowercase.

3. LENGTH()

Find the number of characters in a name.

II. Numeric Function

Numeric Functions perform mathematical operations on numbers.

1. ROUND()

round a number to a specified number of decimal places.

2. ABS()

Returns the absolute value (positive value).

III. Date Function

Date Functions are used to work with date and time values.

1. NOW()

Returns current date and time.

2. CURDATE()

Returns **current date only**.



Window Functions

What is Window Function

Window Function is used to perform calculations across a set of rows related to the current row, without collapsing the result into a single row.

Why Window Functions are used?

Syntax

```
function_name() OVER (  
  PARTITION BY column  
  ORDER BY column  
)
```

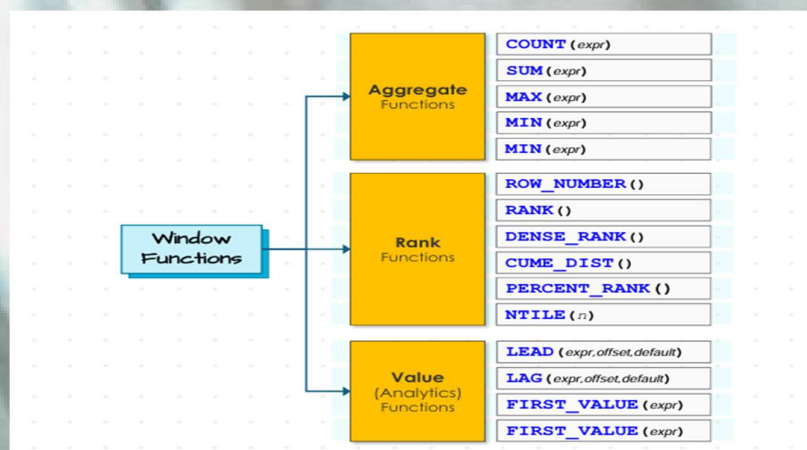
Important Components

1. **OVER():** OVER() is the most important part of a window function in SQL. It defines the window (set of rows) on which the function will operate.
2. **PARTITION BY:-** PARTITION BY is used in a window function to divide the result set into groups (partitions), and the function is applied separately to each group.
3. **ORDER BY:-** ORDER BY is used to define the sequence of rows inside each window (partition) so that calculations are performed in a specific order.

EXAMPLE

```
SELECT empno, ename, deptno, salary,  
RANK() OVER (PARTITION BY deptno ORDER BY salary DESC) AS dept_rank  
FROM Employee;
```

TYPE OF WINDOW FUNCTION



Rank Functions

Rank functions are a type of window function used to assign a rank (position) to each row based on a specified column

1. ROW_NUMBER()

- ROW_NUMBER() is a ranking window function in SQL that assigns a unique sequential number to each row in a result set.

```
SELECT ename, salary,  
ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM Employee;
```

2. RANK()

- RANK() is a window function in SQL that assigns a rank (position) to each row in a result set based on a specified column

```
SELECT ename, salary,  
RANK() OVER (ORDER BY salary DESC) AS rank_num  
FROM Employee;
```

3. DENSE_RANK()

- DENSE_RANK() is a window function used to assign ranks to rows based on a specified column

```
SELECT ename, salary,  
DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_num  
FROM Employee;
```

4. NTILE()

- NTILE() is a window function that divides the result set into a specified number of equal groups (buckets) and assigns a group number to each row.

```
SELECT column_name,  
NTILE(n) OVER (ORDER BY column_name) AS bucket  
FROM table_name;
```

👉 n = number of groups you want

Value Functions

Value functions are window functions used to return values from other rows based on the current row (without collapsing the result set).

1. LEAD()

LEAD() is a window function (value function) used to get the value from the next row in a result set.

```
SELECT column_name,  
       LEAD(column_name) OVER (ORDER BY column_name) AS next_value  
FROM table_name;
```

2. LAG()

LAG() is a window function (value function) used to get the value from the previous row in a result set.

```
SELECT column_name,  
       LAG(column_name) OVER (ORDER BY column_name) AS prev_value  
FROM table_name;
```

3. LAST_VALUE()

LAST_VALUE() is a window function (value function) used to return the last value in an ordered result set.

```
SELECT column_name,  
       LAST_VALUE(column_name) OVER (  
         ORDER BY column_name  
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
       ) AS last_value  
FROM table_name;
```

4. FIRST_VALUE()

FIRST_VALUE() is a window function (value function) that returns the first value in an ordered result set.

```
SELECT column_name,  
       FIRST_VALUE(column_name) OVER (  
         ORDER BY column_name  
       ) AS first_value  
FROM table_name;
```

Normalization

Normalization is the process of organizing data in a database to:

- reduce data duplication (redundancy)
- improve data consistency
- avoid insertion, update, and deletion problems

It divides large tables into smaller related tables using keys.

Types of Normalization

1. First Normal Form (1NF)

1NF means every column should contain only single values and no multiple values should exist in one cell.

It ensures that:

- each column contains only single (atomic) values
- no multiple values are stored in one cell
- each row is unique

Example

```
CREATE TABLE StudentSubjects (  
  StudentID INT,  
  StudentName VARCHAR(50),  
  Subject VARCHAR(50)  
);
```

2. Second Normal Form (2NF)

2NF means the table must be in 1NF and all non-key columns should depend on the complete primary key.

A table is in 2NF if:

- It is already in 1NF
- No partial dependency exists

What is Partial Dependency?

Partial dependency means:

- a column depends on only part of the primary key, not the whole key.

This usually happens when a table has a composite primary key (more than one column as primary key).

Example

Enrollment Table

```
</> SQL  
  
CREATE TABLE Enrollment (  
    StudentID INT,  
    CourseID INT  
);
```

3. Third Normal Form (3NF)

Third Normal Form (3NF) means the table must be in 2NF and non-key columns should depend only on the primary key, not on other non-key columns.

A table is in 3NF if:

- It is already in 2NF
- No transitive dependency exists

What is Transitive Dependency?

Transitive dependency means:

- a non-key column depends on another non-key column instead of the primary key.

4. 4NF (Fourth Normal Form)

4NF removes multi-valued dependency from a table.

A table is in 4NF if:

- it is already in BCNF
- no multi-valued dependency exists

What is Multi-Valued Dependency?

When one column contains multiple independent values.

Example:

A student can have:

- multiple hobbies
- multiple languages

5. 5NF (Fifth Normal Form)

5NF removes join dependency.

A table is in 5NF if:

- it is already in 4NF
- it cannot be divided further without losing data

6. BCNF (Boyce-Codd Normal Form)

BCNF is an advanced form of normalization used to remove redundancy and dependency problems from a table.

A table is in **BCNF** if:

- it is already in **3NF**
- every determinant is a candidate key

Advantages

- Reduces redundancy
- Improves consistency
- Saves storage
- Easier maintenance
- Better database structure

Disadvantages

- More tables
- Complex joins
- Slightly slower queries sometimes

Denormalization

Denormalization is a database optimization technique in which multiple tables are combined into a single table to improve query performance and reduce joins.

Example

```
SELECT s.StudentName, c.CourseName
FROM Student s
JOIN Enrollment e
ON s.StudentID = e.StudentID
JOIN Course c
ON e.CourseID = c.CourseID;
```

Why Use Denormalization?

normalized databases, many joins are required.

Denormalization:

- reduces joins
- increases read speed
- improves performance for reports and analytics

Advantages of Denormalization

- Faster SELECT queries
- Better read performance
- Fewer joins
- Useful for reporting systems
- Improves analytics speed

Disadvantages of Denormalization

- Data redundancy increases
- More storage required
- Update anomalies may occur
- Data inconsistency possible

Difference Between Normalization and Denormalization

Normalization	Denormalization
Reduces redundancy	Increases redundancy
More joins	Fewer joins
Better consistency	Better performance

Indexes & Performance

What is Index?

Index is a database object used to improve the speed of data retrieval operations.

It works like a **book index**:

- Without index → SQL checks every row
- With index → SQL directly finds the data location

Why Use Index?

Indexes improve:

- SELECT query speed
- Searching performance
- Sorting performance

Especially useful on:

- Large tables
- Frequently searched columns

Syntax of INDEX

```
CREATE INDEX index_name  
ON table_name(column_name);
```

Create Index on ename

```
CREATE INDEX idx_ename  
ON Employee(ename);
```


Type of Index?

1. Single Column Index

A Single Column Index is an index created on only one column of a table.

It improves search performance for that specific column.

Used when:

- A column is searched frequently
- Filtering is done on one column

Example

```
SELECT *  
FROM Employee  
WHERE ename = 'Rahul';
```

2. Composite Index

A Composite Index is an index created on multiple columns together.

Used when:

- Queries use more than one column in WHERE
- Data filtering happens on combined columns

Syntax

```
CREATE INDEX index_name  
ON table_name(column1, column2);
```

Example

```
CREATE INDEX idx_name_salary  
ON Employee(ename, salary);
```

3. Unique Index

A Unique Index ensures that duplicate values are not allowed in a column.

Used when:

- Values must remain unique
- Example: email, phone number, Aadhaar number

Syntax

```
CREATE UNIQUE INDEX index_name  
ON table_name(column_name);
```

Example

```
CREATE UNIQUE INDEX idx_email  
ON Employee(email);
```

Difference Between Index Types

Index Type	Columns Used	Duplicate Allowed
Single Column Index	One column	Yes
Composite Index	Multiple columns	Yes
Unique Index	One or more columns	No

What is Query Optimization?

Query optimization is the process of improving SQL query performance and execution speed.

Goal:

- Reduce execution time
- Improve database performance

Example

```
SELECT *  
FROM Employee  
WHERE ename = 'Rahul';
```

CTE(Common Table Expression)

CTE (Common Table Expression) is a temporary named result set created using the WITH keyword to simplify complex SQL queries.

Why Use CTE?

- ✓ Improves readability
- ✓ Simplifies complex queries
- ✓ Reusable in same query
- ✓ Better than long subqueries in many cases

Syntax

```
WITH cte_name AS (  
    SELECT column_name  
    FROM table_name  
)  
SELECT *  
FROM cte_name;
```

Example

```
WITH HighSalary AS (  
    SELECT ename, salary  
    FROM Employee  
    WHERE salary > 45000  
)  
SELECT *  
FROM HighSalary;
```

Recursive CTE in SQL

What is Recursive CTE?

Recursive CTE is a Common Table Expression that repeatedly references itself until a condition is met.

Used for:

- Hierarchical data
- Employee-manager structure
- Tree structure
- Generating numbers

Structure of Recursive CTE

Recursive CTE has 2 parts:

1. **Anchor Query** → starting value
2. **Recursive Query** → repeats itself

Syntax

```
WITH cte_name AS (  
    -- Anchor Query  
    SELECT column_name  
  
    UNION ALL  
  
    -- Recursive Query  
    SELECT column_name  
    FROM cte_name  
    WHERE condition  
)  
SELECT *  
FROM cte_name;
```

Example :- Generate Numbers

```
WITH Numbers AS (  
    SELECT 1 AS num  
  
    UNION ALL  
  
    SELECT num + 1  
    FROM Numbers  
    WHERE num < 5  
)  
SELECT *  
FROM Numbers;
```

Views

What is a View?

A **View** is a virtual table created from one or more tables using a SQL query.

- A view does not store data permanently.
- It displays data from the original table.

Why Use Views?

Views are used to:

- simplify complex queries
- improve security
- show selected data only
- hide sensitive columns

Syntax

```
CREATE VIEW view_name AS  
SELECT column_name  
FROM table_name  
WHERE condition;
```

Example:- View with Condition

```
CREATE VIEW high_salary AS  
SELECT ename, salary  
FROM Employee  
WHERE salary > 45000;
```

Types of Views

1. Simple View

A Simple View is created from only one table.

It:

- **does not use JOIN**
- **does not use GROUP BY**
- **is easy to understand**
- **is often updatable (depends on DB)**

Syntax

```
CREATE VIEW view_name AS  
SELECT column1, column2  
FROM table_name;
```

Example

```
CREATE VIEW emp_view AS
SELECT ename, salary
FROM Employee;
```

2. Complex View

A Complex View is created using:

- multiple tables
- JOIN
- GROUP BY
- Functions

Syntax

```
CREATE VIEW view_name AS
SELECT columns
FROM table1
JOIN table2
ON condition;
```

Example

```
CREATE VIEW emp_dept_view AS
SELECT e.ename, e.salary, d.dname
FROM Employee e
INNER JOIN Department d
ON e.deptno = d.deptno;
```

ER Diagram

ER diagram (short for Entity–Relationship diagram) is a type of diagram used in Database Design to visually represent how data is organized in a database.

Simple explanation

It shows:

- **Entities** → things or objects (like *Student*, *Teacher*, *Book*)
- **Attributes** → details about those entities (like *name*, *ID*, *age*)
- **Relationships** → how entities are connected.

Why ER diagrams are used

- To design databases clearly before building them
- To understand data structure easily
- To show relationships visually

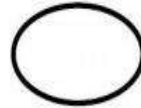
ER Diagram Explanation

_____ 0..n



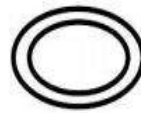
Entity Type

_____ 1..n At least one



Attribute

→ 0..1 At most one



Multi-valued Attribute

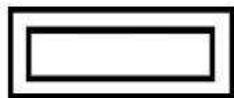
→ 1 Exactly One

sid Underlining attribute: Primary Key

address dotted underling attribute:
discriminator/partial key for a weak entity



Relationship Type

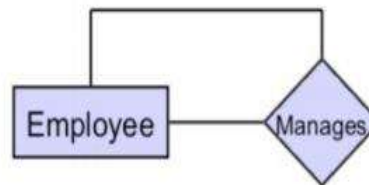


Weak Entity Type (part of)

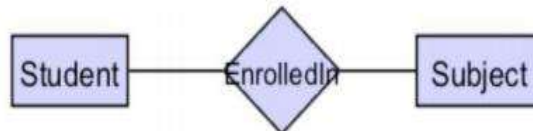


Identifying Relationship Type

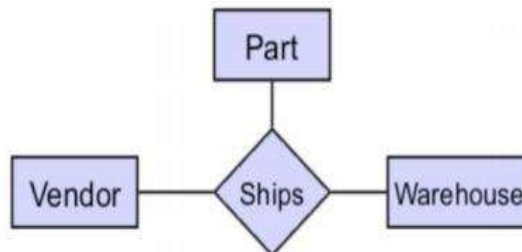
► Unary Relationship (Recursive)



► Binary Relationship (most common kind of relationship)



► Ternary Relationship (three entity types involved)



Cardinality in ER Diagram

- Cardinality tells how many entities can participate in a relationship.

1. One-to-One (1:1)

One record from first table connects with only one record from second table.

Example

- One person → One passport

Person ----- Passport
1 1

2. One-to-Many (1:M)

One record from first table connects with many records from second table.

Example

- One department → Many employees

Department ----- Employee
1 Many

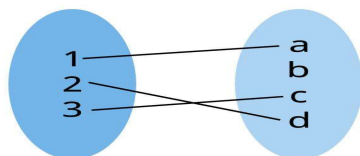
3. Many-to-Many (M:N)

Many records from first table connect with many records from second table.

Example

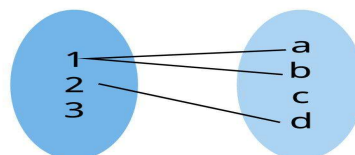
- Many students → Many courses

Student ----- Course
Many Many



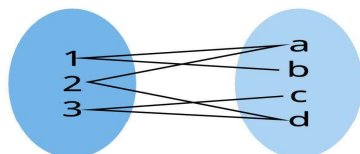
1:1

One to one relationship



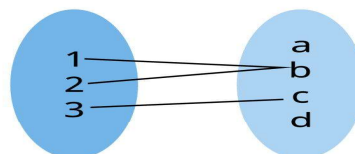
1:N

One to many relationship



M:N

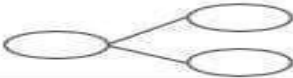
Many to many relationship



N:1

Many to one relationship

Weak Entity

Entity Set 	Strong Entity Set	
	Weak Entity Set	
Attributes 	Simple Attribute	
	Composite Attribute	
	Single-valued Attribute	
	Multivalued Attribute	
	Derived Attribute	
	Null Attribute	
Relationship	Strong Relationship	
	Weak Relationship	

A Weak Entity is an entity that cannot be identified uniquely by its own attributes.

It depends on another entity.

Symbol

- Double Rectangle

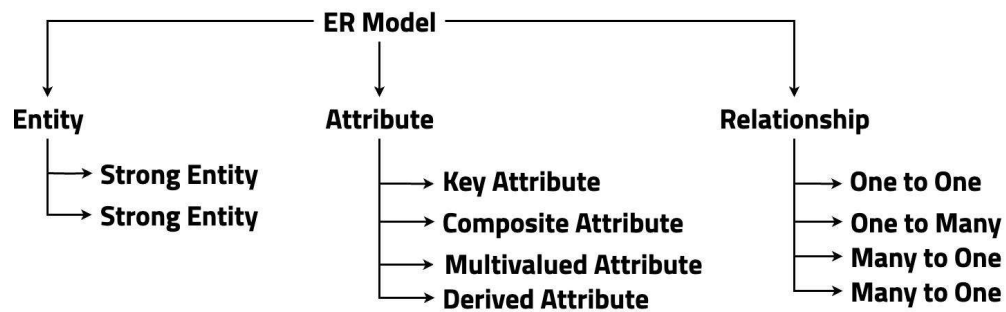
Example

- Employee → Dependent

Dependent cannot exist without Employee.

Employee ----- Dependent

Composite Attribute



A Composite Attribute can be divided into smaller sub-parts.

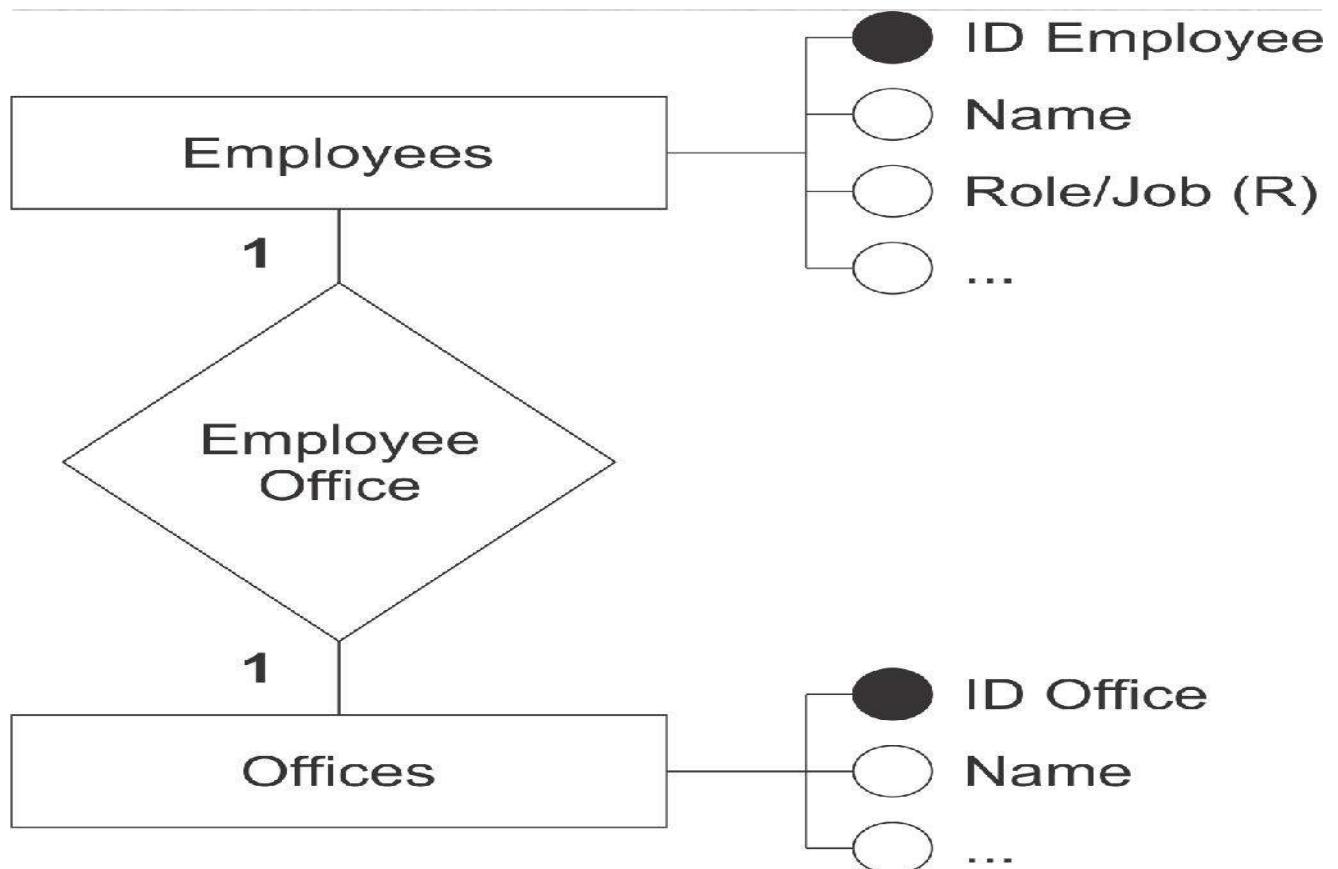
Example

Name can be divided into:

- First Name
- Middle Name
- Last Name

```
Name
├─ First_Name
├─ Middle_Name
└─ Last_Name
```

Multivalued Attribute



A Multivalued Attribute stores multiple values for one entity.

Symbol

- Double Oval

Example

A student can have multiple phone numbers.

```
Student ---- Phone_Number
           (multiple)
```


STORED PROCEDURE

A Stored Procedure is a pre-written SQL query or group of queries stored inside the database that can be executed whenever needed.

Features

- Reusable code
- Improves performance
- Reduces network traffic
- Supports parameters
- Used for business logic

Syntax

```
CREATE PROCEDURE procedure_name
AS
BEGIN
    -- SQL Statements
END
```

Example

Create Table

```
CREATE TABLE Employee(
    emp_id INT,
    emp_name VARCHAR(50),
    salary INT
);
```

Stored Procedure

```
CREATE PROCEDURE ShowEmployee
AS
BEGIN
    SELECT * FROM Employee;
END;
```

Execute Procedure

```
EXEC ShowEmployee;
```

Procedure with Parameter

```
CREATE PROCEDURE GetEmployee
    @id INT
AS
BEGIN
    SELECT *
    FROM Employee
    WHERE emp_id = @id;
END;
```

Execute

```
EXEC GetEmployee 101;
```

FUNCTION(User Defined Function - UDF)

User Defined Function (UDF) is a function created by the user in SQL Server to perform calculations or return data.

Function:

- Accepts parameters
- Performs operations
- Returns a value or table

Why Use Function?

- Reuse code
- Reduce repeated logic
- Improve readability
- Perform calculations easily

Types of UDF

1. Scalar Function
2. Table Valued Function (TVF)

1. Scalar Function

A **Scalar Function** is a type of **User Defined Function (UDF)** in SQL that returns **only one single value**.

Syntax

```
CREATE FUNCTION function_name
(
    @parameter datatype
)
RETURNS datatype
AS
BEGIN
    RETURN value
END
```

Example 1: Addition Function

```
CREATE FUNCTION AddNumbers
(
    @a INT,
    @b INT
)
RETURNS INT
AS
BEGIN
    RETURN @a + @b
END
```

Execute Function

```
SELECT dbo.AddNumbers(10,20) AS Result;
```

2. Table Valued Function (TVF)

Table Valued Function (TVF) is a type of **User Defined Function (UDF)** that returns a **table** instead of a single value.

Types of TVF

1. Inline Table Valued Function

Returns a table directly using one SELECT statement.

Syntax

```
CREATE FUNCTION function_name()
RETURNS TABLE
AS
RETURN
(
    SELECT column_name
    FROM table_name
)
```

Example

```
CREATE TABLE Employee(
    emp_id INT,
    emp_name VARCHAR(50),
    salary INT
);
```

2. Multi-Statement Table Valued Function

Uses multiple SQL statements and returns a table variable.

Syntax

```
CREATE FUNCTION function_name()  
RETURNS @table_variable TABLE  
(  
    column_name datatype  
)  
AS  
BEGIN  
  
    -- SQL Statements  
  
    RETURN  
END
```

Example

```
CREATE FUNCTION EmployeeData()  
RETURNS @EmpTable TABLE  
(  
    id INT,  
    name VARCHAR(50)  
)  
AS  
BEGIN  
  
    INSERT INTO @EmpTable  
    SELECT emp_id, emp_name  
    FROM Employee;  
  
    RETURN  
END
```